

Notes on Binary Classification

Support Vector Machines and Logistic Regression

DSBA Mathematics Refresher 2026

Abstract

This session is about **binary classification**: given a set of labelled examples, we want to build a rule that predicts the label of a new, unseen example. We will see two classical methods, which are also two applications of what we have done in the previous sessions:

- **Support Vector Machines**, which lead to a *constrained* optimization problem (Constrained Optimization session);
- **Logistic regression**, which leads to an *unconstrained* optimization problem, solved by gradient descent (Session 3).

1 The Binary Classification Problem

Our data is a set of N labelled points

$$\{(\vec{\mathbf{x}}_k, y_k)\}_{k=1}^N$$

where $\vec{\mathbf{x}}_k \in \mathbb{R}^n$ describes the k -th example (its "features") and y_k is its **label**. As there are only two classes, the label takes only two values, and it is convenient to write them

$$y_k \in \{-1, +1\}$$

Example: $\vec{\mathbf{x}}_k$ could be $\begin{pmatrix} \text{income} \\ \text{age} \end{pmatrix}$ of a customer, and y_k whether that customer subscribed (+1) or not (-1). Given a new customer, we want to predict whether they will subscribe.

We look for a rule of the form: compute some function $h(\vec{\mathbf{x}})$, and predict +1 if $h(\vec{\mathbf{x}}) > 0$, and -1 if $h(\vec{\mathbf{x}}) < 0$. The set $\{h(\vec{\mathbf{x}}) = 0\}$ is the **decision boundary**.

2 Separating Hyperplanes

The simplest choice is a **linear** function:

$$h(\vec{\mathbf{x}}) = \vec{\mathbf{w}} \cdot \vec{\mathbf{x}} - b$$

where $\vec{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$ are the parameters to be learned. In $\mathbb{R}^2$, $\{h(\vec{x}) = 0\}$ is a line; in $\mathbb{R}^3$ it is a plane; in general it is called a **hyperplane**. It cuts the space into two regions:

$$\mathcal{R}_{-1} = \{\vec{x} : \vec{w} \cdot \vec{x} - b < 0\} \quad \text{and} \quad \mathcal{R}_{+1} = \{\vec{x} : \vec{w} \cdot \vec{x} - b > 0\}$$

Handling the two labels at once

Classifying the point $\vec{x}_k$ correctly means

$$h(\vec{x}_k) > 0 \text{ if } y_k = +1 \quad \text{and} \quad h(\vec{x}_k) < 0 \text{ if } y_k = -1$$

In both cases $h(\vec{x}_k)$ has the *same sign* as y_k , so their product is positive, and the two conditions collapse into a single one:

$$y_k h(\vec{x}_k) > 0$$

This trick — multiplying by the label ± 1 to replace a case distinction by one inequality — is used constantly, and it is the reason for labelling the classes -1 and $+1$ rather than 0 and 1 .

Distance to the hyperplane

The vector $\vec{w}$ is perpendicular to the hyperplane. Therefore, if $\vec{x}_1$ is any point of the hyperplane (so that $\vec{w} \cdot \vec{x}_1 = b$), the distance from a point $\vec{x}_0$ to the hyperplane is the length of the projection of $\vec{x}_0 - \vec{x}_1$ onto the unit normal $\frac{\vec{w}}{\|\vec{w}\|}$:

$$d(\vec{x}_0) = \left| (\vec{x}_0 - \vec{x}_1) \cdot \frac{\vec{w}}{\|\vec{w}\|} \right| = \frac{|\vec{w} \cdot \vec{x}_0 - b|}{\|\vec{w}\|}$$

Note that this does not depend on which point $\vec{x}_1$ was chosen.

Example: In $\mathbb{R}^2$ with $\vec{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $b = \frac{3}{2}$, the hyperplane is the line $x + y = \frac{3}{2}$, and the distance from the point $(1, 1)$ to it is

$$\frac{|1 + 1 - \frac{3}{2}|}{\sqrt{1^2 + 1^2}} = \frac{1/2}{\sqrt{2}} = \frac{\sqrt{2}}{4} \approx 0.354$$

3 Support Vector Machines

When the data is **linearly separable**, there are usually *infinitely many* hyperplanes separating the two classes. Which one should we choose?

The idea of the Support Vector Machine is to pick the one that leaves as much room as possible on both sides: the one **maximizing the margin**.

The margin

Notice that h and $2h$ define the same decision boundary: multiplying $\vec{w}$ and b by the same positive number changes nothing. We can therefore fix the scale by

requiring that the closest points satisfy $|h(\vec{x}_k)| = 1$. The picture is then made of three regions:

$$\mathcal{R}_{-1} : \vec{w} \cdot \vec{x} - b < -1, \quad \mathcal{R}_0 : |\vec{w} \cdot \vec{x} - b| < 1, \quad \mathcal{R}_{+1} : \vec{w} \cdot \vec{x} - b > 1$$

where $\mathcal{R}_0$ is the **margin band**, which should contain no data point at all.

Its width is computed as follows: let $\vec{x}_+$ and $\vec{x}_-$ be two points, one on each edge, with $\vec{x}_+ - \vec{x}_- = d \cdot \frac{\vec{w}}{\|\vec{w}\|}$ perpendicular to both edges. Subtracting the two equations $\vec{w} \cdot \vec{x}_+ - b = 1$ and $\vec{w} \cdot \vec{x}_- - b = -1$:

$$\vec{w} \cdot (\vec{x}_+ - \vec{x}_-) = 2 \quad \implies \quad d \|\vec{w}\| = 2 \quad \implies \quad d = \frac{2}{\|\vec{w}\|}$$

The optimization problem

Requiring every point to be correctly classified *and* outside the margin band is written, with the same trick as before,

$$y_k (\vec{w} \cdot \vec{x}_k - b) \geq 1 \quad \text{for all } k$$

and maximizing the margin $\frac{2}{\|\vec{w}\|}$ means minimizing $\|\vec{w}\|$. The SVM is therefore the solution of

$$\text{minimize } \frac{1}{2} \|\vec{w}\|^2 \quad \text{subject to} \quad y_k (\vec{w} \cdot \vec{x}_k - b) \geq 1 \quad \text{for all } k$$

We minimize $\frac{1}{2} \|\vec{w}\|^2$ rather than $\|\vec{w}\|$ because it has the same minimizer and is much nicer to differentiate.

This is exactly a constrained optimization problem with inequality constraints, of the kind studied in the Constrained Optimization session: it is solved with Lagrange multipliers (one per data point) and slack variables. The details are outside the scope of this course — in practice, we let a computer solve it.

Support vectors

At the optimum, most constraints are *inactive* (their multiplier is 0): the corresponding points are strictly outside the margin band and play no role. The points for which $y_k h(\vec{x}_k) = 1$ — those lying exactly on the edges of the band — are the **support vectors**, and they alone determine the solution. Moving or deleting any other point does not change the answer. This is where the method gets its name, and it is also what makes it economical: the model is described by a handful of points.

4 Non-Separable Data: the Kernel Trick

Real data is rarely linearly separable. Two classical answers:

- **Soft margin:** allow a few points to violate the constraints, at a price added to the cost function.

- **Kernel trick:** send the data into a bigger space through a non-linear map ϕ , and look for a hyperplane *there*.

Example: Take the points $(0, \pm 1)$ and $(\pm 1, 0)$ in the class $+1$, and $(0, \pm 2)$ and $(\pm 2, 0)$ in the class -1 . The first four are on the circle of radius 1, the last four on the circle of radius 2: no line can separate them, since one class surrounds the other.

Now apply

$$\phi(x, y) = (x^2 + y^2, x^2 - y^2)$$

The four positive points are all sent to points with first coordinate 1, and the four negative points to points with first coordinate 4. In these new coordinates (u, v) , the vertical line $u = \frac{5}{2}$ separates them perfectly.

Coming back to the original coordinates, $u = \frac{5}{2}$ means $x^2 + y^2 = \frac{5}{2}$: the separating "line" of the new space is a **circle** in the original one. A linear separation after transformation is a non-linear separation before it.

5 Logistic Regression

The SVM outputs a hard decision: $+1$ or -1 . Often we would rather have a **probability**: "this customer subscribes with probability 0.8". This is what logistic regression provides.

The model

We keep a linear function of the features, but we squash it into the interval $(0, 1)$ with the **sigmoid** function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

so that the model reads (here with labels written 0 and 1, which is the usual convention for this method)

$$P(Y = 1 \mid X = x) = \sigma(\beta_0 + \beta_1 x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

The sigmoid is increasing, tends to 0 at $-\infty$ and to 1 at $+\infty$, and $\sigma(0) = \frac{1}{2}$. The decision boundary — where the model hesitates between the two classes — is therefore $\beta_0 + \beta_1 x = 0$.

It also satisfies two identities that we will use constantly:

$$\sigma(-z) = 1 - \sigma(z) \quad \text{and} \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

The first says $P(Y = 0) = 1 - P(Y = 1)$, as it must; the second is what makes the gradient below so simple.

Why the sigmoid? Odds and log-odds

The sigmoid is not an arbitrary way of squashing a number into $(0, 1)$. Recall that the **odds** of an event of probability p are $\frac{p}{1-p}$ (a probability of 0.75 gives odds of 3, i.e. "3 to 1"). If $p = \sigma(z)$, then, using $1 - p = \sigma(-z)$:

$$\frac{p}{1-p} = e^z \quad \text{that is} \quad \log\left(\frac{p}{1-p}\right) = z = \beta_0 + \beta_1 x$$

So logistic regression is a *linear* model — not of the probability, but of the **log-odds**. This is what one should remember, for two reasons:

- it explains the shape of the model: a quantity that grows without bound (the log-odds) is converted into one that saturates (a probability);
- it is how the coefficients are read in practice. Adding one to x adds β_1 to the log-odds, hence **multiplies the odds by** e^{β_1} — a factor that does not depend on where we started. A coefficient $\beta_1 = 0.64$ means "one extra unit multiplies the odds by $e^{0.64} \approx 1.9$ ", which is a sentence a client can act on, whereas " $\beta_1 = 0.64$ " is not.

The cost function: log loss

To fit β_0 and β_1 , we need to measure how wrong the model is. The usual choice is the **log loss** (also called cross-entropy):

$$\mathcal{L}(\beta_0, \beta_1) = - \sum_{k=1}^N \left[y_k \log(p_k) + (1 - y_k) \log(1 - p_k) \right], \quad \text{where } p_k = \sigma(\beta_0 + \beta_1 x_k)$$

Since y_k is 0 or 1, exactly one of the two terms survives for each point, and the contribution of the point is

$$- \log(\text{probability the model assigned to the correct answer})$$

It is 0 for a perfect prediction, and grows to $+\infty$ as the model becomes confidently wrong. This last property is what makes the log loss much more suitable than the squared error here: it punishes confident mistakes very severely.

Minimizing the cost

Unlike the linear regression of Session 3, this minimization has no closed-form solution: we use **gradient descent**. The gradient takes a remarkably simple form:

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = \sum_{k=1}^N (p_k - y_k) \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \beta_1} = \sum_{k=1}^N x_k (p_k - y_k)$$

(the term $p_k - y_k$ is simply the error made on the k -th point), and the parameters are updated with

$$\beta_0 \leftarrow \beta_0 - \eta \frac{\partial \mathcal{L}}{\partial \beta_0} \quad \text{and} \quad \beta_1 \leftarrow \beta_1 - \eta \frac{\partial \mathcal{L}}{\partial \beta_1}$$

where η is the learning rate.

Note that at the minimum the gradient vanishes, which gives the two identities

$$\sum_k p_k = \sum_k y_k \quad \text{and} \quad \sum_k x_k p_k = \sum_k x_k y_k$$

They are a convenient way of checking a numerical result.

Everything extends to more features

With $\vec{\mathbf{x}} \in \mathbb{R}^n$ instead of a single number x , the model becomes

$$P(Y = 1 \mid \vec{\mathbf{x}}) = \sigma(\vec{\mathbf{w}} \cdot \vec{\mathbf{x}} + b)$$

and all the formulas above hold, with x_k replaced by $\vec{\mathbf{x}}_k$. The decision boundary $\vec{\mathbf{w}} \cdot \vec{\mathbf{x}} + b = 0$ is again a hyperplane: like the SVM, logistic regression is a *linear* classifier — the two methods differ in what they optimize, not in the shape of their boundary.